HA TRADING LAB · COMPANION TO THE COMPLETE GUIDE

How Every Part Works

Kafka, RabbitMQ, Redis, PostgreSQL, PgBouncer, HAProxy and our own services, explained one card at a time: what each one is, what it does in the lab, how it keeps working when a copy fails, and what you will see if you switch it off.

3 copies, majority rules
Every stateful part has a copy in each zone and decides by 2-of-3 vote.

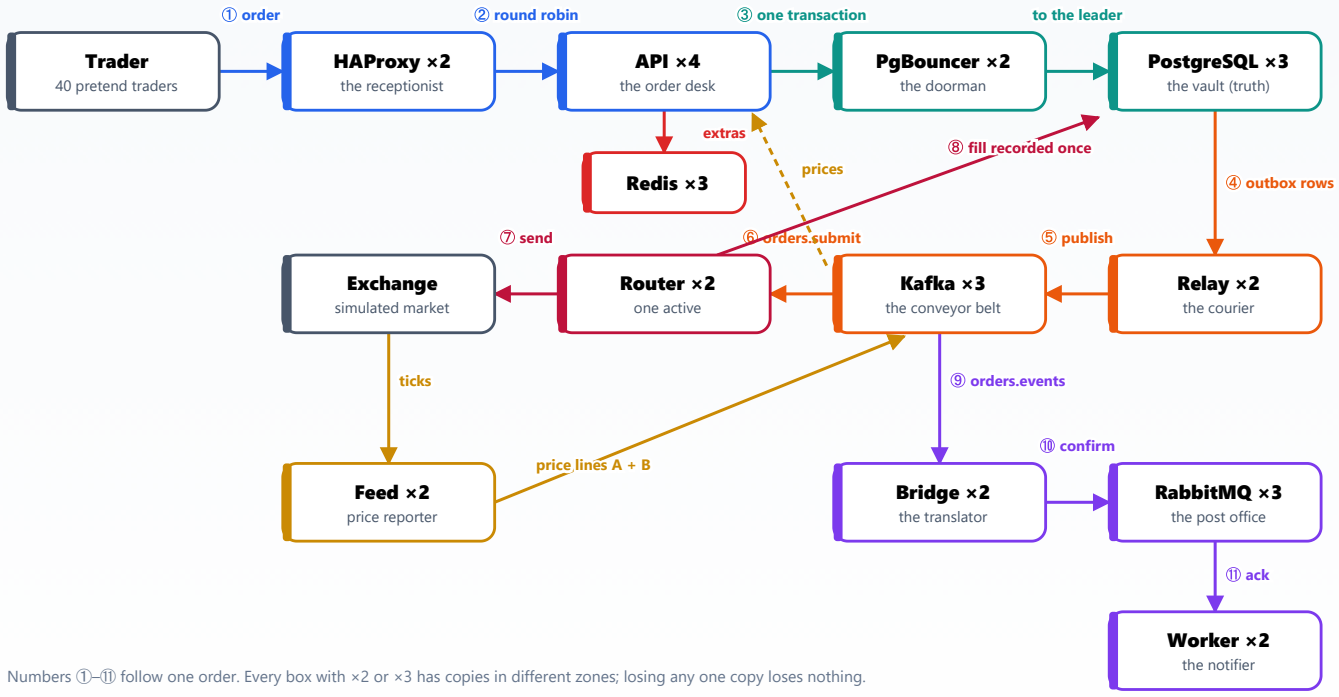
Retry until confirmed
Every hand-over is confirmed; anything unconfirmed is sent again.

Drop duplicates
Unique ids everywhere, so a message sent twice counts once.

1 The big picture
2 The front door
3 Storing data safely
4 Moving messages without losing them
5 Notifications: Bridge, RabbitMQ, Worker
6 Talking to the market
7 Running the lab
8 Side by side

1 The big picture

The lab is a small copy of a real trading back end: 38 containers spread over three pretend data centres (Azure zone 1, Azure zone 2, Oracle Mumbai). This picture follows one order from the trader's click to the notification that it was filled. Every later page explains one box.



Numbers ①–⑪ follow one order. Every box with ×2 or ×3 has copies in different zones; losing any one copy loses nothing.

One order through every part. Blue: front door · green: data · orange: Kafka · purple: RabbitMQ · red: router.

1 · Three copies, majority rules

Every system that stores something has 3 copies, one per zone, and decides by majority (2 of 3). One zone can always be lost.

2 · Retry until confirmed

Every message is sent again until the receiver confirms it. It may arrive twice, never zero times.

3 · Drop duplicates

Every receiver recognises a repeat by its unique id (order ref, fill id, event id), so "twice" counts once.

How to read the cards: each card says what the part is (with an everyday comparison), what it does in our lab, how it keeps working when a copy fails, and what you will see if you kill it. Times were measured in the lab's own failure drills.

2 The front door

HAProxy

`haproxy-1 · haproxy-2`

The receptionist who sends each visitor to a free desk.

A load balancer: it receives every request and forwards it to one healthy server.

IN OUR LAB

- Spreads requests over api-1…api-4 in turn (round robin).
- Asks each API `/health` every 1 s; 2 failures in a row = down, 2 successes = back.
- An API that is shutting down answers 503 first, so it is drained without errors.

HOW IT STAYS ALIVE

- Two HAProxys in different zones; traders know both addresses.
- A request that hits a dead API is retried on another one (`retries 2` , `redispatch`).

If it dies: traders see a short pause and reconnect to the other HAProxy (~1 s). A dead API is skipped within ~2 s.

Code and config: `config/haproxy-edge.cfg`

API servers

`api-1 … api-4`

The order desk that writes your order into the ledger.

Our Python service that traders talk to: login, quotes, place and cancel orders.

IN OUR LAB

- **Login:** a signed token (account · expiry · signature); any API can check it alone.
- **Place order:** ONE database transaction saves the order, reserves the cash and writes 2 outbox rows, then answers **202 Received**.
- **No double orders:** each order has a `client_ref` ; the database refuses a second copy.
- **Prices:** reads both Kafka price lines and keeps the newest; refuses market orders on prices older than 3 s.

HOW IT STAYS ALIVE

- Four identical copies; none keeps anything important in memory.
- Redis problems never stop trading: a circuit breaker skips Redis for 5 s at a time.

If it dies: HAProxy stops sending to it within ~2 s; traders retry with the same `client_ref` . Measured ~2 s, 0 lost.

Code and config: `app/api.py` · `app/common.py`

3 Storing data safely

PgBouncer

`pgbouncer-1 · pgbouncer-2`

The doorman who lets many guests share a few seats.

A connection pool: hundreds of app connections share a few real database connections.

IN OUR LAB

- Transaction pooling: a database connection is lent for one transaction, then returned (15 per pool).
- A small HAProxy next to it asks every database node `/primary` every 0.5 s; only the leader says yes, so PgBouncer always points at the leader.

HOW IT STAYS ALIVE

- Two PgBouncers; the apps' connection string lists both, so the driver tries the next one.
- When the leader changes, connections move to the new one within ~0.5 s; the apps never know.

If it dies: connection refused → the driver uses the other PgBouncer. Measured < 1 s, nothing noticed by traders.

Code and config: `images/pgbouncer/pgbouncer.ini` · `images/pgbouncer/haproxy-pg.cfg`

PostgreSQL + Patroni + etcd

`pg-1 … pg-3 · etcd-1 … etcd-3`

The bank vault with two live copies and a referee.

The source of truth: accounts, orders, fills, positions, the outbox and the router lease.

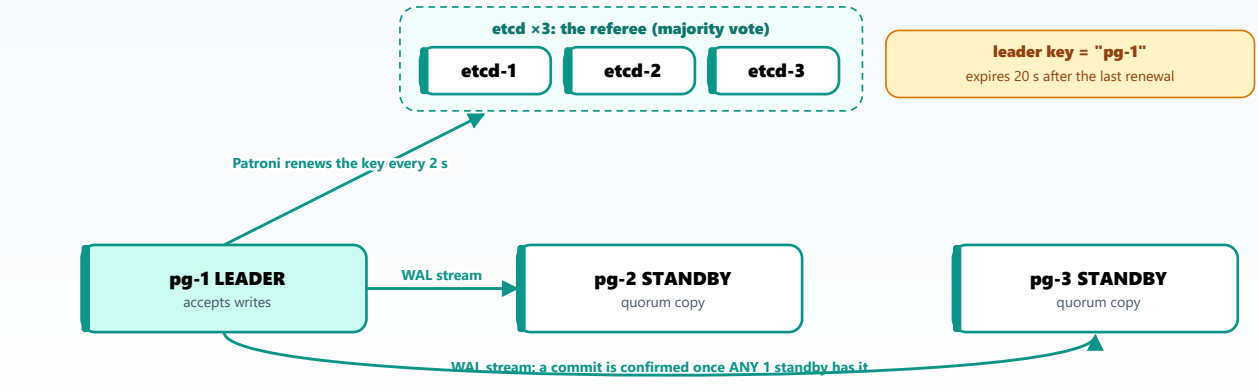
IN OUR LAB

- **1 leader** takes writes; **2 standbys** copy every change (the WAL stream).
- **Quorum commit:** a write is confirmed only when at least one standby has it: "Received" means stored on 2 machines.
- **Patroni** manages each node and renews the leader key in **etcd**, a 3-node majority-vote store.
- **Backups:** pgBackRest full backup daily, incremental every 30 min, every change log archived → restore to any second.

HOW IT STAYS ALIVE

- The leader key lives 20 s. If the leader stops renewing it, a standby with every confirmed write takes it.
- A returning old leader is rewound (`pg_rewind`) and rejoins as a standby, never a second leader.
- Two nodes down: no majority, so it refuses writes rather than guess.

If it dies: orders get "try again" for ~20 s, then the new leader takes over. Measured 16–21 s, 0 lost.



If pg-1 dies, its key expires; the standby with every confirmed write takes the key and becomes leader. Two dead nodes = no majority = the database waits safely.

Who is leader is decided by etcd's majority, not by the database nodes themselves.

Code and config: `images/postgres/patroni.yml` · `images/postgres/pgbackrest.conf` · `app/schema.sql`

Redis + Sentinel

`redis-1 … redis-3 · sentinel-1 … sentinel-3`

A fast whiteboard, with three guards who replace it if it breaks.

An in-memory store: very fast, used for things that are nice to have.

IN OUR LAB

- Holds **optional extras only** (rate-limit counters, caches), never money or orders.
- 1 master + 2 replicas; every change is also written to disk (AOF), so a restart keeps the data.

HOW IT STAYS ALIVE

- 3 Sentinels ping the master; silent for 3 s and **2 of 3 agree** → they promote the best replica.
- The APIs ask Sentinel "who is master now?"; a restarted Redis asks too and rejoins as a replica.

If it dies: a new master in 4–5 s; meanwhile the APIs simply skip Redis. Trading never stops.

Code and config: `config/redis-entry.sh` · `config/sentinel-entry.sh`

4 Moving messages without losing them

Outbox + Relay

`relay-1 · relay-2`

The courier who only ticks a parcel off once it is signed for.

The pattern that guarantees every saved order also gets announced, even if a server crashes in between.

IN OUR LAB

- The API writes the message into the `outbox` table in the same transaction as the order.
- The relay reads unsent rows, sends them to Kafka, and marks them sent only after Kafka confirms.
- Both relays run; `FOR UPDATE SKIP LOCKED` gives each row to only one of them.

HOW IT STAYS ALIVE

- If a relay dies mid-way, the row is still unsent and the other relay sends it (maybe twice: receivers drop repeats).
- If Kafka is down, rows wait safely in the database and go out when it is back.

If it dies: nothing visible: the twin keeps publishing. 0 lost.



Why not send to Kafka straight from the API? A crash between "saved" and "sent" would lose the message.

Code and config: `app/relay.py`

Kafka

`kafka-1 · kafka-2 · kafka-3`

A conveyor belt that keeps everything on it, so anyone can re-read it.

A durable, ordered log of messages. Messages stay on disk; each reader keeps its own position.

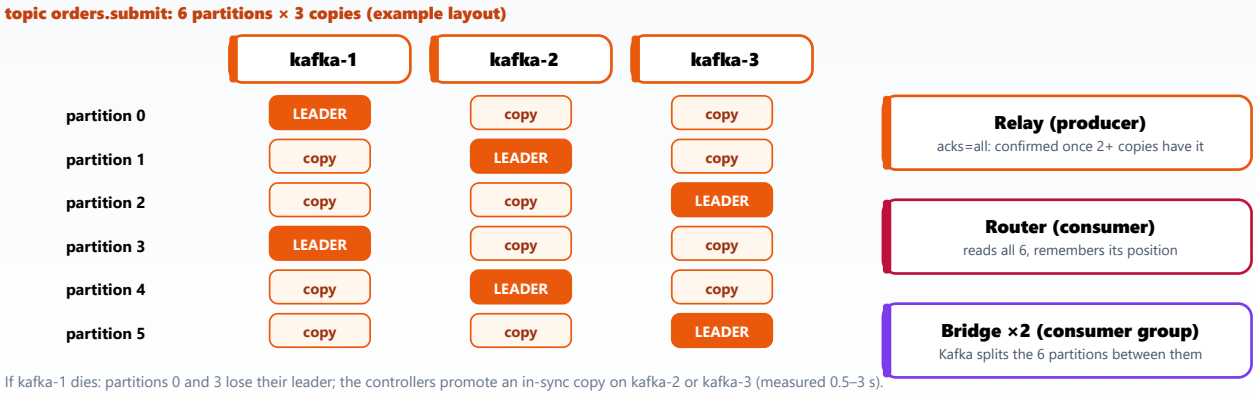
IN OUR LAB

- `orders.submit` : orders for the router · `orders.events` : received / submitted / filled.
- Both: 6 partitions (lanes), 3 copies each, every lane has a leader broker.
- `acks=all` + `min.insync.replicas=2` : confirmed only when 2 brokers have it; idempotent producers.
- Prices: two independent lines (`lineA` on kafka-1, `lineB` on kafka-3); the APIs read both.

HOW IT STAYS ALIVE

- KRaft: the 3 brokers also vote on leaders. A broker silent for 2 s loses its lanes to in-sync copies.
- Readers resume from their saved position, so nothing is skipped.

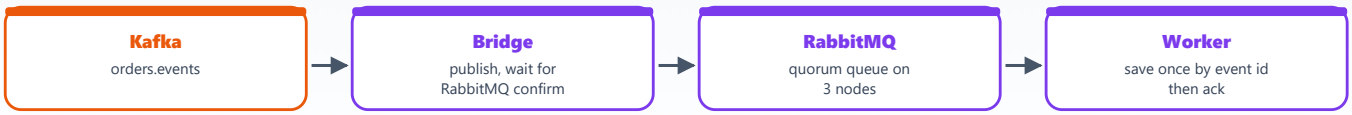
If it dies: its lanes get new leaders in 0.5–3 s; prices keep flowing from the other line (gap < 0.2 s). 0 lost.



Each lane has one leader and two copies on other brokers; producers and readers always talk to the leader.

Code and config: `docker-compose.yml (Kafka settings)` · `app/common.py`

5 Notifications: Bridge, RabbitMQ, Worker



The notification path: every hand-over is confirmed before the sender forgets the message.

Bridge

`bridge-1 · bridge-2`

The translator between the conveyor belt and the post office.

Reads order events from Kafka and publishes them to RabbitMQ.

IN OUR LAB

- Waits for RabbitMQ's **publisher confirm** before saving its Kafka position.
- The two bridges form one consumer group: Kafka splits the partitions between them.

HOW IT STAYS ALIVE

- A crash before the confirm means the event is sent again, never skipped.
- If one bridge dies, Kafka gives its partitions to the other.

If it dies: notifications arrive a few seconds late; 0 missing.

Code and config: `app/bridge.py`

RabbitMQ

`rabbit-1 · rabbit-2 · rabbit-3`

The post office: each letter goes to one postman and is signed for.

A message broker for **tasks**: each message goes to one worker and is deleted once handled (Kafka, by contrast, keeps a replayable log).

IN OUR LAB

- Quorum queue `notifications` stored on all 3 nodes; a leader is elected by majority (Raft).
- Workers must **ack**; an un-acked message is delivered again.
- Poison messages: after 5 failed deliveries a message goes to `notifications.dlq` for a human.

HOW IT STAYS ALIVE

- A dead node's queue leader moves to another node within one status poll.
- `pause_minority`: a node cut off from the other two pauses itself, so the queue never splits in two.

If it dies: queue leader moves in < 1 s; clients reconnect to another node. 0 lost.

Code and config: `config/rabbitmq.conf` · `config/rabbit-entry.sh`

Worker

`worker-1 · worker-2`

The postman who delivers the letter and gets the signature.

Takes a notification from RabbitMQ and records it as delivered to the trader.

IN OUR LAB

- Saves it to the `notifications` table; the event id is the key, so a repeat is ignored.
- Then acks; takes up to 20 messages at a time (prefetch).

HOW IT STAYS ALIVE

- Two competing workers on one queue: if one dies, its un-acked messages go to the other.

If it dies: nothing visible; the other worker continues.

Code and config: `app/worker.py`

6 Talking to the market

Order router

`router-1 · router-2 (one active)`

The single trader allowed at the exchange counter.

The only service that sends orders to the exchange and records the fills.

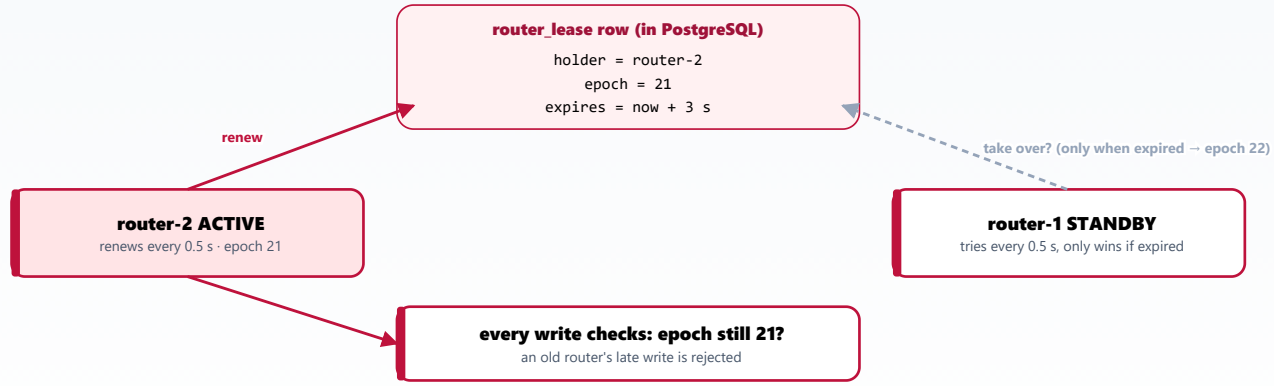
IN OUR LAB

- Reads `orders.submit`, claims the order, sends it to the exchange.
- Reads the exchange's execution stream; records each fill **once** (the exchange's fill id is the database key).
- Cash, position, reservation, event and its own reading position are saved in the **same transaction**.
- End of session: cancels orders the exchange no longer holds and releases the cash.

HOW IT STAYS ALIVE

- Only one is active: a **lease** in PostgreSQL with an **epoch** number (fencing token).
- The new active router first **reconciles**: asks the exchange what it already has, by `client_ref`.
- A sweep re-sends orders stuck as PENDING for more than 1.5 s.

If it dies: orders wait ~3 s, then the standby takes over. Measured 1.4–4 s, 0 lost, 0 sent twice.



A router that has not renewed for 2 s stops acting by itself; the other takes over after the 3 s lease ends and first reconciles with the exchange.

Lease + epoch: two routers can never act at the same time, even if the old one wakes up late.

Code and config: `app/router.py`

Feed

`feed-1 · feed-2`

The reporter who reads the price board and announces it.

Long-polls the exchange for new prices (about 20 a second) and writes them to Kafka.

IN OUR LAB

- feed-1 writes price line A, feed-2 writes line B: two independent paths.
- Each tick carries the exchange's boot id and sequence number.

HOW IT STAYS ALIVE

- If the exchange restarts (new boot id) or goes silent for 2 s, the feed resyncs.
- The APIs keep the highest (boot, seq) per stock, so duplicates between the lines are dropped.

If it dies: the other line keeps prices flowing; gap < 0.2 s.

Code and config: `app/feed.py`

Exchange (simulated)

exchange

A pretend stock exchange, lab only.

Stands in for NSE: matches orders, produces prices and reports fills. It is never connected to IBKR or any real market.

IN OUR LAB

- Order ids and fill ids include a boot id, so they are unique across restarts.
- Offers an audit endpoint so the lab can check every fill reached our database.

HOW IT STAYS ALIVE

- One copy (it plays the outside world). Our side reconciles with it after any failure.

If it dies: prices pause ~3 s; on restart the router cancels orders the exchange forgot and releases the cash.

Code and config: `app/exchange.py`

7 Running the lab

Control

`control (dashboard :9700)`

The lab manager with a clipboard and a set of switches.

Drives traffic, checks correctness and lets you break things on purpose.

IN OUR LAB

- **40 pretend traders**: buy, sell, cancel; retry with the same order ref and back-off with jitter.
- **Audit every 3 s**: lost orders, duplicates at the exchange, missing fills, wrong cash or positions, missing notifications.
- **Dashboard** at http://127.0.0.1:9700 with Stop / Kill / Pause / Zone down / Restore buttons and an event timeline.
- **Auto-heal** restarts dead containers after 15 s (like Kubernetes); **backups**: incremental every 30 min, full daily.

HOW IT STAYS ALIVE

- It only watches: if control dies, trading and every failover carry on without it.

If it dies: the dashboard goes away; the trading system is unaffected.

Code and config: `app/control.py` · `app/ui/index.html`

8 Side by side

Kafka, RabbitMQ and Redis side by side

	Kafka	RabbitMQ	Redis
Think of it as	conveyor belt / log	post office	whiteboard
After reading	message stays; anyone can re-read	message is deleted once acked	value stays until overwritten
Best for	event streams, orders, prices, replay	tasks that one worker must do once	fast counters and caches
In our lab	orders.submit, orders.events, price lines A/B	notifications (+ dead-letter queue)	rate limits, caches (optional)
Copies	3 per partition, confirmed on 2	quorum queue on 3 nodes	1 master + 2 replicas
Who picks a new leader	KRaft controllers (vote)	Raft inside the queue (vote)	Sentinels (2 of 3 agree)
Measured recovery	0.5–3 s	< 1 s	4–5 s

Who notices a failure, and who decides

Part fails	Who notices	Who decides the new route	Measured
API server	HAProxy health check	HAProxy alone	~2 s
PgBouncer	the app's database driver	the driver (tries the next host)	< 1 s
PostgreSQL leader	Patroni on the standbys	etcd majority	16–21 s
etcd member	the other etcd nodes	the 2 survivors vote	~2 s
Kafka broker	KRaft controllers	controller majority	0.5–3 s
RabbitMQ node	the queue's other members	Raft vote	< 1 s
Redis master	Sentinels	2 of 3 Sentinels	4–5 s
Active router	the standby router	the lease in PostgreSQL	1.4–4 s
Relay, feed, bridge, worker	nobody needs to	the twin is already running	0 s
A whole zone	all of the above at once	the other two zones' majorities	all recovered, 0 lost

One line each

Part	In one line
HAProxy	sends each request to a healthy server
API	saves the order and its messages in one transaction
PgBouncer	shares a few database connections among many apps, always pointing at the leader
PostgreSQL	the truth: money, orders, fills, on 2 machines before "Received"
Patroni + etcd	a majority vote decides which database node is leader
Redis + Sentinel	fast extras; Sentinels vote in a new master; trading never depends on it
Outbox + relay	the message is saved with the order, so it cannot be lost
Kafka	durable, replayable belt for orders, events and prices
Router	exactly one at the exchange; old ones are fenced out; fills recorded once
Bridge → RabbitMQ → Worker	notifications handed over with a confirmation at every step
Control	traffic, audit, dashboard and the switches to break things